Technical Architecture Specification-Intelligent Core Banking & Commercial Credit Risk Engine

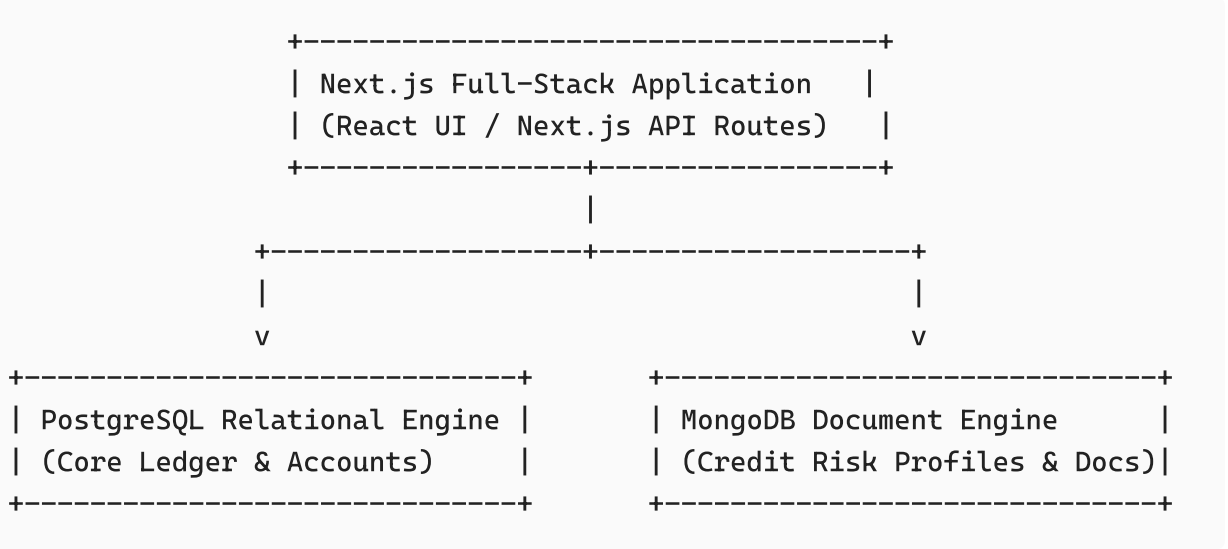
---

Technical Architecture Specification: Intelligent Core Banking & Commercial Credit Risk Engine

1. System Architecture & Workload Isolation

Hybrid Database Model Justification

Modern financial institutions operate at the intersection of two conflicting data management requirements: strict transactional consistency for accounting ledgers, and flexible schema adaptability for credit underwriting. Standard enterprise applications often attempt to force both paradigms into a single database system, creating severe trade-offs in operational integrity or system maintainability. The Intelligent Core Banking & Commercial Credit Risk Engine utilizes a hybrid database architecture that isolates transactional OLTP workloads within PostgreSQL while offloading complex, dynamic risk analytics to MongoDB.



Core banking platforms require immutable double-entry bookkeeping, absolute precision in balance calculations, and robust protection against concurrency anomalies. PostgreSQL fulfills these requirements through its implementation of strict ANSI-SQL standards, Write-Ahead Logging (WAL), Multi-Version Concurrency Control (MVCC), and serializable isolation levels. Financial transactions demand that debits and credits balance to zero across distinct accounts within a single atomic unit of work. The relational model enforces these business rules at the engine level through foreign key constraints, explicit row-level locking, and transactional triggers, ensuring that phantom reads, non-repeatable reads, and race conditions are prevented during funds transfers.

Conversely, commercial credit risk evaluation requires processing unstructured and semi-structured payloads. Corporate balance sheets, seasonal cash-flow projections, income statements, external tax returns, and credit bureau JSON payloads vary across business sectors, entity sizes, and regulatory jurisdictions. Attempting to model these dynamic data structures in a relational schema results in fragmented Entity-Attribute-Value (EAV) designs or complex schema migrations whenever a bureau updates its payload structure. MongoDB provides a schema-less document model that stores dynamic, deeply nested JSON profiles natively. This design allows underwriting algorithms to run rich aggregation pipelines directly over nested cash-flow arrays without incurring the performance overhead of multi-table relational joins.

Operational Parameter	Core Banking Engine (PostgreSQL)	Commercial Credit Engine (MongoDB)
Domain Responsibility	General ledger, customer accounts, double-entry transactions, loan amortization schedules, audit logs.	Financial profiles, cash-flow JSONs, income statements, tax documents, credit score payloads.
Data Structure	Highly normalized 3NF relational tables with strict type enforcement.	Polymorphic, nested BSON documents accommodating Retail, SME, and Corporate structures.
Concurrency Guarantees	Strict ACID compliance via MVCC, row-level locks (`FOR UPDATE`), and write-ahead logs.	Eventual consistency across nodes, document-level isolation, and targeted transactional sessions.
Primary Query Workload	High-concurrency point queries, atomic update operations, transaction logging.	Dynamic aggregation pipelines, array filtering, ratio calculation, dynamic risk scoring.
Integrity Enforcement	Foreign key constraints, unique indexes, database-level CHECK constraints, triggers.	JSON Schema validation rules, application-level schema enforcement, dynamic indexing.

Data Flow Topology & Request Routing

Requests flow through the Next.js full-stack application layer, which acts as the API gateway and orchestration engine. Role-Based Access Control (RBAC) middleware inspects incoming JSON Web Tokens (JWT) or session cookies to authenticate the identity and verify the authorizations of users—categorized into `customer`, `loan_officer`, and `risk_admin` roles. Once authorized, API route handlers direct data operations along dedicated database channels based on the target domain.

The transactional path routes all funds management, account modifications, and repayment actions directly to a PostgreSQL connection pool. These operations execute inside explicit transaction blocks (`BEGIN` ... `COMMIT`), where row locks protect balance variables from

concurrent access. Every balance update triggers synchronous, database-level logging to produce an immutable, append-only audit trail.

The credit analytics path directs all profile evaluations, financial statement ingestions, and risk scoring actions to a MongoDB connection pool. Dynamic aggregations process income statements, current liabilities, and cash-flow series natively within MongoDB, returning computed metrics such as Debt-Service Coverage Ratios (DSCR) and Operating Cash Flow (OCF) margins directly to the application server.

When a loan application passes underwriting review, an event orchestrator executes a cross-database transaction flow. The system writes the final risk decision and approved parameters to MongoDB, while simultaneously invoking a PostgreSQL PL/pgSQL function to generate the formal amortization schedule and open the corresponding loan ledger.

2. Comprehensive Relational Schema (PostgreSQL)

The relational engine uses a normalized 3NF relational schema designed for absolute data integrity, performance, and compliance. It uses ANSI SQL standard types, primary keys generated via Universally Unique Identifiers (UUID v4), explicit foreign key references with restricted deletion policies, and dynamic CHECK constraints.

SQL

```
-- PostgreSQL Core Banking Relational Schema DDL
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- Enumerated Role-Based Access Control and System States
CREATE TYPE user_role AS ENUM ('customer', 'loan_officer', 'risk_admin');
CREATE TYPE account_type AS ENUM ('checking', 'savings', 'escrow',
'loan_disbursement');
CREATE TYPE account_status AS ENUM ('active', 'frozen', 'closed');
CREATE TYPE loan_status AS ENUM ('pending', 'under_review', 'approved',
'rejected', 'active', 'closed', 'defaulted');
CREATE TYPE transaction_type AS ENUM ('deposit', 'withdrawal', 'transfer',
'loan_disbursement', 'repayment');

-- 1. Identity & Access Management Table
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    role user_role NOT NULL DEFAULT 'customer',
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. Core Accounts Ledger Table
CREATE TABLE accounts (
    account_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
    account_number VARCHAR(34) UNIQUE NOT NULL,
    type account_type NOT NULL,
    status account_status NOT NULL DEFAULT 'active',
    currency VARCHAR(3) NOT NULL DEFAULT 'USD',
    balance DECIMAL(15, 2) NOT NULL DEFAULT 0.00,
    minimum_balance DECIMAL(15, 2) NOT NULL DEFAULT 0.00,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_balance_min_threshold CHECK (balance >= minimum_balance),
    CONSTRAINT chk_positive_min_balance CHECK (minimum_balance >= 0.00)
);

-- 3. Double-Entry General Ledger Transactions Table
CREATE TABLE transactions (
    transaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reference_number VARCHAR(64) UNIQUE NOT NULL,
    source_account_id UUID REFERENCES accounts(account_id) ON DELETE
RESTRICT,
    destination_account_id UUID REFERENCES accounts(account_id) ON DELETE
RESTRICT,
    type transaction_type NOT NULL,
    amount DECIMAL(15, 2) NOT NULL,
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_transaction_amount_positive CHECK (amount > 0.00),
    CONSTRAINT chk_distinct_accounts CHECK (
        (type = 'deposit' AND source_account_id IS NULL AND
destination_account_id IS NOT NULL) OR
        (type = 'withdrawal' AND source_account_id IS NOT NULL AND
destination_account_id IS NULL) OR
        (type IN ('transfer', 'loan_disbursement', 'repayment') AND
source_account_id IS NOT NULL AND destination_account_id IS NOT NULL AND
source_account_id <> destination_account_id)
    )
);

-- 4. Commercial Credit Loans Table
CREATE TABLE loans (
    loan_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    borrower_id UUID NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
    officer_id UUID REFERENCES users(user_id) ON DELETE SET NULL,
    mongodb_credit_profile_id VARCHAR(24) NOT NULL,
    principal_amount DECIMAL(15, 2) NOT NULL,
    annual_interest_rate DECIMAL(5, 4) NOT NULL,
    term_months INT NOT NULL,
```

```
    status loan_status NOT NULL DEFAULT 'pending',
    approved_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_loan_principal_positive CHECK (principal_amount > 0.00),
    CONSTRAINT chk_interest_rate_bounds CHECK (annual_interest_rate >=
0.0000 AND annual_interest_rate <= 1.0000),
    CONSTRAINT chk_term_positive CHECK (term_months > 0)
);

-- 5. Amortization Schedule & Repayments Table
CREATE TABLE loan_repayments (
    repayment_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    loan_id UUID NOT NULL REFERENCES loans(loan_id) ON DELETE CASCADE,
    installment_number INT NOT NULL,
    due_date DATE NOT NULL,
    principal_due DECIMAL(15, 2) NOT NULL,
    interest_due DECIMAL(15, 2) NOT NULL,
    total_due DECIMAL(15, 2) NOT NULL,
    amount_paid DECIMAL(15, 2) NOT NULL DEFAULT 0.00,
    is_settled BOOLEAN NOT NULL DEFAULT FALSE,
    paid_at TIMESTAMPTZ,
    CONSTRAINT chk_installment_positive CHECK (installment_number > 0),
    CONSTRAINT chk_repayment_components CHECK (total_due = principal_due +
interest_due),
    CONSTRAINT uq_loan_installment UNIQUE (loan_id, installment_number)
);

-- 6. Immutable Ledger Audit Log Table
CREATE TABLE account_audit_log (
    audit_id BIGSERIAL PRIMARY KEY,
    account_id UUID NOT NULL REFERENCES accounts(account_id) ON DELETE
CASCADE,
    old_balance DECIMAL(15, 2) NOT NULL,
    new_balance DECIMAL(15, 2) NOT NULL,
    delta_amount DECIMAL(15, 2) NOT NULL,
    operation_type VARCHAR(32) NOT NULL,
    changed_by_app_user VARCHAR(255) DEFAULT CURRENT_USER,
    changed_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Performance & Audit Optimizations (Indexes)
CREATE INDEX idx_users_role_email ON users(role, email);
CREATE INDEX idx_accounts_user_id ON accounts(user_id);
CREATE INDEX idx_accounts_status ON accounts(status) WHERE status =
'active';
CREATE INDEX idx_transactions_source_dest ON transactions(source_account_id,
destination_account_id);
CREATE INDEX idx_transactions_created_at ON transactions(created_at DESC);
CREATE INDEX idx_loans_borrower_status ON loans(borrower_id, status);
```

```
CREATE INDEX idx_loan_repayments_schedule ON loan_repayments(loan_id,
due_date, is_settled);
CREATE INDEX idx_audit_account_timestamp ON account_audit_log(account_id,
changed_at DESC);
```

3. Advanced Relational Logic & Transactional Mechanics

Dynamic Amortization Schedule Generation

The loan engine computes fixed periodic amortization schedules using the standard annuity payment formula. Given an initial principal amount P, a periodic interest rate $r = \frac{\text{annual_rate}}{12}$, and a total number of repayment periods n, the fixed periodic payment M is calculated as:

$$M = P \cdot \frac{r(1+r)^n}{(1+r)^n - 1}$$

The PL/pgSQL function below calculates payments dynamically, generates installment rows in the `loan_repayments` table, handles rounding logic on the final installment, and transitions the loan status from `approved` to `active` within a single atomic execution.

SQL

```
CREATE OR REPLACE FUNCTION fn_generate_amortization_schedule(
    p_loan_id UUID
) RETURNS VOID AS $$
DECLARE
    v_principal NUMERIC(15, 2);
    v_annual_rate NUMERIC(5, 4);
    v_term_months INT;
    v_status loan_status;

    v_monthly_rate NUMERIC(15, 10);
    v_monthly_payment NUMERIC(15, 2);
    v_remaining_balance NUMERIC(15, 2);
    v_interest_due NUMERIC(15, 2);
    v_principal_due NUMERIC(15, 2);

    v_due_date DATE := CURRENT_DATE;
    i INT;
BEGIN
    -- Fetch loan parameters and lock row
    SELECT principal_amount, annual_interest_rate, term_months, status
    INTO v_principal, v_annual_rate, v_term_months, v_status
    FROM loans
    WHERE loan_id = p_loan_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Loan contract ID % not found.', p_loan_id;
```

```
    END IF;

    IF v_status <> 'approved' THEN
        RAISE EXCEPTION 'Amortization generation rejected: Loan status is
%', v_status;
    END IF;

    -- Clear pre-existing un-settled schedules if re-calculating
    DELETE FROM loan_repayments WHERE loan_id = p_loan_id AND is_settled =
FALSE;

    v_remaining_balance := v_principal;

    IF v_annual_rate = 0 THEN
        v_monthly_payment := ROUND(v_principal / v_term_months, 2);
        v_monthly_rate := 0;
    ELSE
        v_monthly_rate := v_annual_rate / 12.0;
        v_monthly_payment := ROUND(
            v_principal * (v_monthly_rate * POWER(1 + v_monthly_rate,
v_term_months)) /
            (POWER(1 + v_monthly_rate, v_term_months) - 1),
            2
        );
    END IF;

    -- Iteratively compute monthly installments
    FOR i IN 1..v_term_months LOOP
        v_due_date := v_due_date + INTERVAL '1 month';

        IF i = v_term_months THEN
            -- Adjust final payment to account for floating-point rounding
discrepancies
            v_principal_due := v_remaining_balance;
            v_interest_due := ROUND(v_remaining_balance * v_monthly_rate,
2);
            v_monthly_payment := v_principal_due + v_interest_due;
        ELSE
            v_interest_due := ROUND(v_remaining_balance * v_monthly_rate,
2);
            v_principal_due := v_monthly_payment - v_interest_due;
        END IF;

        INSERT INTO loan_repayments (
            repayment_id,
            loan_id,
            installment_number,
            due_date,
            principal_due,
            interest_due,
```

```
            total_due,
            amount_paid,
            is_settled
        ) VALUES (
            gen_random_uuid(),
            p_loan_id,
            i,
            v_due_date,
            v_principal_due,
            v_interest_due,
            v_monthly_payment,
            0.00,
            FALSE
        );

        v_remaining_balance := v_remaining_balance - v_principal_due;
    END LOOP;

    -- Mark loan as active
    UPDATE loans
    SET status = 'active', updated_at = CURRENT_TIMESTAMP
    WHERE loan_id = p_loan_id;

END;
$$ LANGUAGE plpgsql;
```

Immutable Ledger Triggers & Balance Validation

Automated Account Audit Trigger

Financial regulations mandate an immutable record of all balance modifications. The following trigger automatically records every `UPDATE` on the `accounts` table into an append-only audit trail.

SQL

```
CREATE OR REPLACE FUNCTION fn_audit_account_balance_change()
RETURNS TRIGGER AS $$
BEGIN
    IF (OLD.balance IS DISTINCT FROM NEW.balance) THEN
        INSERT INTO account_audit_log (
            account_id,
            old_balance,
            new_balance,
            delta_amount,
            operation_type,
            changed_by_app_user,
            changed_at
        ) VALUES (
```

```
            NEW.account_id,
            OLD.balance,
            NEW.balance,
            (NEW.balance - OLD.balance),
            TG_OP,
            CURRENT_USER,
            CURRENT_TIMESTAMP
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_account_balance_audit
AFTER UPDATE ON accounts
FOR EACH ROW
EXECUTE FUNCTION fn_audit_account_balance_change();
```

Minimum Balance Threshold Trigger

To prevent unauthorized overdrafts, this trigger executes prior to updates, validating that the account balance remains above the required threshold.

SQL

```
CREATE OR REPLACE FUNCTION fn_enforce_minimum_balance()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.balance < NEW.minimum_balance THEN
        RAISE EXCEPTION 'Transaction Rejected: Account % balance (%) falls
below required minimum threshold (%)',
            NEW.account_id, NEW.balance, NEW.minimum_balance
            USING ERRCODE = '23514'; -- check_violation
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validate_account_minimum_balance
BEFORE UPDATE ON accounts
FOR EACH ROW
EXECUTE FUNCTION fn_enforce_minimum_balance();
```

Atomic Funds Transfer Procedure with Lock Ordering

To execute atomic transfers securely across accounts, funds transfers must use explicit row locking (`FOR UPDATE`). Acquiring locks in a deterministic order (by sorted UUID) prevents deadlocks during concurrent transfers between identical account pairs.

SQL

```
CREATE OR REPLACE PROCEDURE sp_transfer_funds(
    p_source_account_id UUID,
    p_destination_account_id UUID,
    p_amount NUMERIC(15, 2),
    p_description TEXT,
    OUT p_transaction_id UUID
)
LANGUAGE plpgsql AS $$
DECLARE
    v_source_balance NUMERIC(15, 2);
    v_source_min_balance NUMERIC(15, 2);
    v_source_status account_status;
    v_dest_status account_status;
    v_first_lock UUID;
    v_second_lock UUID;
BEGIN
    IF p_amount <= 0 THEN
        RAISE EXCEPTION 'Transfer amount must be strictly positive';
    END IF;

    IF p_source_account_id = p_destination_account_id THEN
        RAISE EXCEPTION 'Source and destination accounts must be distinct';
    END IF;

    -- Enforce lock ordering by UUID string value to eliminate deadlocks
    IF p_source_account_id < p_destination_account_id THEN
        v_first_lock := p_source_account_id;
        v_second_lock := p_destination_account_id;
    ELSE
        v_first_lock := p_destination_account_id;
        v_second_lock := p_source_account_id;
    END IF;

    -- Acquire explicit row-level locks
    PERFORM account_id FROM accounts WHERE account_id = v_first_lock FOR
UPDATE;
    PERFORM account_id FROM accounts WHERE account_id = v_second_lock FOR
UPDATE;

    -- Validate source account state
    SELECT balance, minimum_balance, status
    INTO v_source_balance, v_source_min_balance, v_source_status
    FROM accounts WHERE account_id = p_source_account_id;

    IF v_source_status <> 'active' THEN
        RAISE EXCEPTION 'Source account % is not active',
p_source_account_id;
```

```
    END IF;

    IF (v_source_balance - p_amount) < v_source_min_balance THEN
        RAISE EXCEPTION 'Insufficient funds: Available balance % is less
than transfer amount % plus minimum threshold %',
            v_source_balance, p_amount, v_source_min_balance;
    END IF;

    -- Validate destination account state
    SELECT status INTO v_dest_status
    FROM accounts WHERE account_id = p_destination_account_id;

    IF v_dest_status <> 'active' THEN
        RAISE EXCEPTION 'Destination account % is not active',
p_destination_account_id;
    END IF;

    -- Execute double-entry ledger update
    UPDATE accounts
    SET balance = balance - p_amount, updated_at = CURRENT_TIMESTAMP
    WHERE account_id = p_source_account_id;

    UPDATE accounts
    SET balance = balance + p_amount, updated_at = CURRENT_TIMESTAMP
    WHERE account_id = p_destination_account_id;

    -- Record transaction entry
    p_transaction_id := gen_random_uuid();
    INSERT INTO transactions (
        transaction_id,
        reference_number,
        source_account_id,
        destination_account_id,
        type,
        amount,
        description
    ) VALUES (
        p_transaction_id,
        'TXN-' || UPPER(SUBSTRING(p_transaction_id::text FROM 1 FOR 8)),
        p_source_account_id,
        p_destination_account_id,
        'transfer',
        p_amount,
        p_description
    );

EXCEPTION
    WHEN OTHERS THEN
        -- Re-raise exception to trigger an automatic transaction rollback
        RAISE;
```

```
END;
$$;
```

Analytical Database Views

Database views provide pre-aggregated reporting for executive decision-making, ensuring that analytics queries do not alter underlying transaction tables.

SQL

```
-- View 1: Executive Portfolio Summary
CREATE OR REPLACE VIEW vw_executive_portfolio_summary AS
SELECT
    COUNT(DISTINCT a.account_id) AS total_active_accounts,
    SUM(a.balance) AS aggregate_deposits,
    AVG(a.balance) AS average_account_balance,
    COALESCE(SUM(l.principal_amount) FILTER (WHERE l.status = 'active'),
0.00) AS total_active_loan_exposure,
    COALESCE(SUM(l.principal_amount) FILTER (WHERE l.status = 'defaulted'),
0.00) AS total_defaulted_exposure,
    COUNT(l.loan_id) FILTER (WHERE l.status = 'pending') AS
pending_loan_applications,
    ROUND(
        (COUNT(l.loan_id) FILTER (WHERE l.status = 'defaulted')::NUMERIC /
        NULLIF(COUNT(l.loan_id) FILTER (WHERE l.status IN ('active',
'closed', 'defaulted')), 0)) * 100, 2
    ) AS non_performing_loan_ratio_percentage
FROM accounts a
LEFT JOIN loans l ON a.user_id = l.borrower_id;

-- View 2: Loan Officer Underwriting Performance
CREATE OR REPLACE VIEW vw_officer_loan_performance AS
SELECT
    u.user_id AS officer_id,
    CONCAT(u.first_name, ' ', u.last_name) AS officer_name,
    COUNT(l.loan_id) AS total_processed_loans,
    COUNT(l.loan_id) FILTER (WHERE l.status = 'approved' OR l.status =
'active') AS approved_loans,
    COUNT(l.loan_id) FILTER (WHERE l.status = 'rejected') AS rejected_loans,
    COALESCE(SUM(l.principal_amount) FILTER (WHERE l.status IN ('approved',
'active')), 0.00) AS total_approved_capital,
    COALESCE(AVG(l.annual_interest_rate) FILTER (WHERE l.status IN
('approved', 'active')), 0.00) AS weighted_avg_approved_rate
FROM users u
LEFT JOIN loans l ON u.user_id = l.officer_id
WHERE u.role = 'loan_officer'
GROUP BY u.user_id, u.first_name, u.last_name;
```

4. MongoDB Document Design & Aggregation Pipelines

JSON Schema Validation (`credit_profiles` Collection)

The `credit_profiles` collection uses MongoDB BSON Schema validation to enforce structural constraints while allowing flexibility for dynamic financial payloads. It accommodates varying data structures across Retail, Small and Medium Enterprise (SME), and Corporate borrowers.

JSON

```
{
  "$jsonSchema": {
    "bsonType": "object",
    "required": ["borrower_id", "entity_type", "legal_name",
"tax_identifier", "financial_statements"],
    "properties": {
      "_id": { "bsonType": "objectId" },
      "borrower_id": { "bsonType": "string", "description": "Foreign UUID
referencing PostgreSQL users.user_id" },
      "entity_type": { "enum": ["RETAIL", "SME", "CORPORATE"] },
      "legal_name": { "bsonType": "string" },
      "tax_identifier": { "bsonType": "string" },
      "credit_bureau_payload": {
        "bsonType": "object",
        "required": ["bureau_name", "score", "delinquencies_24m"],
        "properties": {
          "bureau_name": { "bsonType": "string" },
          "score": { "bsonType": "int" },
          "delinquencies_24m": { "bsonType": "int" }
        }
      },
      "financial_statements": {
        "bsonType": "array",
        "items": {
          "bsonType": "object",
          "required": [
            "fiscal_year", "fiscal_quarter", "revenue",
"operating_expenses",
            "net_operating_income", "operating_cash_flow",
"total_debt_service",
            "short_term_liabilities", "total_assets"
          ],
          "properties": {
            "fiscal_year": { "bsonType": "int" },
            "fiscal_quarter": { "bsonType": "int" },
            "revenue": { "bsonType": "decimal" },
            "operating_expenses": { "bsonType": "decimal" },
            "net_operating_income": { "bsonType": "decimal" },
```

```
            "operating_cash_flow": { "bsonType": "decimal" },
            "total_debt_service": { "bsonType": "decimal" },
            "short_term_liabilities": { "bsonType": "decimal" },
            "total_assets": { "bsonType": "decimal" }
          }
        }
      },
      "updated_at": { "bsonType": "date" }
    }
  }
}
```

Multi-Stage Risk Analytics Aggregation Pipelines

Pipeline 1: Debt-Service Coverage Ratio (DSCR) Pipeline

Debt-Service Coverage Ratio measures an entity's ability to cover its debt obligations using net operating income:

$$\text{DSCR} = \frac{\text{Net Operating Income}}{\text{Total Debt Service}}$$

The aggregation pipeline filters the target borrower, flattens historical financial statements, isolates the Trailing Twelve Months (TTM) period, and calculates the resulting ratio.

JavaScript

```
db.credit_profiles.aggregate([
  { $match: { borrower_id: "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11" } },
  { $unwind: "$financial_statements" },
  { $sort: { "financial_statements.fiscal_year": -1,
"financial_statements.fiscal_quarter": -1 } },
  { $limit: 4 },
  {
    $group: {
      _id: "$_id",
      legal_name: { $first: "$legal_name" },
      ttm_noi: { $sum: "$financial_statements.net_operating_income" },
      ttm_debt_service: { $sum: "$financial_statements.total_debt_service" }
    }
  },
  {
    $project: {
      legal_name: 1,
      ttm_noi: 1,
      ttm_debt_service: 1,
      dscr: {
        $cond: [
          { $eq: ["$ttm_debt_service", NumberDecimal("0.00")] },
```

```
          NumberDecimal("999.99"),
          { $divide: ["$ttm_noi", "$ttm_debt_service"] }
        ]
      }
    }
  }
]);
```

Pipeline 2: Operating Cash Flow (OCF) Margin Analytics

Operating Cash Flow Margin measures operational efficiency by evaluating cash generated relative to total revenue:

$$\text{OCF Margin} = \frac{\text{Operating Cash Flow}}{\text{Total Revenue}}$$

JavaScript

```
db.credit_profiles.aggregate([
  { $unwind: "$financial_statements" },
  {
    $group: {
      _id: { borrower_id: "$borrower_id", year:
"$financial_statements.fiscal_year" },
      legal_name: { $first: "$legal_name" },
      annual_ocf: { $sum: "$financial_statements.operating_cash_flow" },
      annual_revenue: { $sum: "$financial_statements.revenue" }
    }
  },
  {
    $project: {
      borrower_id: "$_id.borrower_id",
      fiscal_year: "$_id.year",
      legal_name: 1,
      annual_ocf: 1,
      annual_revenue: 1,
      ocf_margin_percentage: {
        $multiply: [
          {
            $cond: [
              { $eq: ["$annual_revenue", NumberDecimal("0.00")] },
              0,
              { $divide: ["$annual_ocf", "$annual_revenue"] }
            ]
          },
          100
        ]
      }
    }
```

```
  },
  { $sort: { fiscal_year: -1, ocf_margin_percentage: -1 } }
]);
```

Pipeline 3: Composite Credit Risk Scoring Engine

This multi-stage pipeline aggregates bureau data, cash flows, and balance sheet metrics to compute a credit score (scaled from 0 to 100) and assign a corresponding risk tier.

JavaScript

```
db.credit_profiles.aggregate([
  { $unwind: "$financial_statements" },
  { $sort: { "financial_statements.fiscal_year": -1,
"financial_statements.fiscal_quarter": -1 } },
  {
    $group: {
      _id: "$_id",
      borrower_id: { $first: "$borrower_id" },
      legal_name: { $first: "$legal_name" },
      bureau_score: { $first: "$credit_bureau_payload.score" },
      delinquencies: { $first: "$credit_bureau_payload.delinquencies_24m" },
      latest_noi: { $first: "$financial_statements.net_operating_income" },
      latest_debt_service: { $first:
"$financial_statements.total_debt_service" },
      latest_assets: { $first: "$financial_statements.total_assets" },
      latest_liabilities: { $first:
"$financial_statements.short_term_liabilities" }
    }
  },
  {
    $project: {
      borrower_id: 1,
      legal_name: 1,
      bureau_score: 1,
      dscr: {
        $cond: [
          { $eq: ["$latest_debt_service", NumberDecimal("0.00")] },
          2.5,
          { $divide: ["$latest_noi", "$latest_debt_service"] }
        ]
      },
      current_ratio: {
        $cond: [
          { $eq: ["$latest_liabilities", NumberDecimal("0.00")] },
          3.0,
          { $divide: ["$latest_assets", "$latest_liabilities"] }
        ]
      },
```

```
      delinquencies: 1
    }
  },
  {
    $project: {
      borrower_id: 1,
      legal_name: 1,
      metrics: {
        dscr: "$dscr",
        current_ratio: "$current_ratio",
        bureau_score: "$bureau_score"
      },
      calculated_risk_score: {
        $add: [
          { $multiply: [{ $divide: ["$bureau_score", 850] }, 30] },
          {
            $multiply: [
              { $cond: [{ $gt: ["$dscr", 2.0] }, 1.0, { $divide: ["$dscr",
2.0] }] },
              40
            ]
          },
          {
            $multiply: [
              { $cond: [{ $gt: ["$current_ratio", 2.0] }, 1.0, { $divide:
["$current_ratio", 2.0] }] },
              30
            ]
          },
          { $multiply: ["$delinquencies", -15] }
        ]
      }
    }
  },
  {
    $project: {
      borrower_id: 1,
      legal_name: 1,
      metrics: 1,
      final_score: { $max: [0, { $min: [100, "$calculated_risk_score"] }] },
      risk_tier: {
        $switch: {
          branches: [
            { case: { $gte: ["$calculated_risk_score", 80] }, then:
"LOW_RISK_PRIME" },
            { case: { $gte: ["$calculated_risk_score", 60] }, then:
"MODERATE_RISK" },
            { case: { $gte: ["$calculated_risk_score", 40] }, then:
"SUBPRIME_HIGH_MONITORING" }
          ],
```

```
          default: "CRITICAL_DEFAULT_RISK"
        }
      }
    }
  }
]);
```

5. Next.js Web Interface & API Integration

Connection Pool Management Architecture

To operate efficiently within Next.js serverless and API route environments, database connections must be managed using cached singletons. This approach prevents connection exhaustion during bursts of HTTP traffic.

TypeScript

```
// lib/postgres.ts - PostgreSQL Connection Pool Singleton
import { Pool } from 'pg';

declare global {
  var postgresPool: Pool | undefined;
}

export const pgPool =
  global.postgresPool ||
  new Pool({
    host: process.env.PGHOST,
    port: parseInt(process.env.PGPORT || '5432'),
    user: process.env.PGUSER,
    password: process.env.PGPASSWORD,
    database: process.env.PGDATABASE,
    max: 20,
    idleTimeoutMillis: 30000,
    connectionTimeoutMillis: 2000,
  });

if (process.env.NODE_ENV !== 'production') global.postgresPool = pgPool;
```

TypeScript

```
// lib/mongodb.ts - MongoDB Driver Singleton
import { MongoClient } from 'mongodb';

const uri = process.env.MONGODB_URI || 'mongodb://localhost:27017';
let client: MongoClient;
let clientPromise: Promise<MongoClient>;
```

```
declare global {
  var mongoClientPromise: Promise<MongoClient> | undefined;
}

if (process.env.NODE_ENV === 'development') {
  if (!global.mongoClientPromise) {
    client = new MongoClient(uri);
    global.mongoClientPromise = client.connect();
  }
  clientPromise = global.mongoClientPromise;
} else {
  client = new MongoClient(uri);
  clientPromise = client.connect();
}

export default clientPromise;
```

Dual-Database Next.js Route Handler

The route handler below orchestrates cross-database operations upon loan approval. It fetches the credit profile from MongoDB, updates the loan state in PostgreSQL, generates the amortization schedule, and updates MongoDB with the approval audit log.

TypeScript

```
// app/api/loans/approve/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { pgPool } from '@/lib/postgres';
import clientPromise from '@/lib/mongodb';
import { ObjectId } from 'mongodb';

export async function POST(req: NextRequest) {
  const body = await req.json();
  const { loanId, officerId, mongoProfileId } = body;

  const pgClient = await pgPool.connect();

  try {
    // Fetch MongoDB Credit Profile
    const mongoClient = await clientPromise;
    const db = mongoClient.db(process.env.MONGODB_DBNAME);
    const profile = await db.collection('credit_profiles').findOne({ _id:
new ObjectId(mongoProfileId) });

    if (!profile) {
      return NextResponse.json({ error: 'Target credit profile not found' },
{ status: 404 });
    }
```

```
    // Execute PostgreSQL Transaction
    await pgClient.query('BEGIN');

    const updateResult = await pgClient.query(
      `UPDATE loans
       SET status = 'approved', officer_id = $1, approved_at = NOW(),
updated_at = NOW()
       WHERE loan_id = $2 AND status = 'pending'
       RETURNING borrower_id, principal_amount`,
      [officerId, loanId]
    );

    if (updateResult.rowCount === 0) {
      await pgClient.query('ROLLBACK');
      return NextResponse.json({ error: 'Loan is not in a pending state' },
{ status: 400 });
    }

    // Execute Stored Procedure for Amortization Schedule
    await pgClient.query('SELECT fn_generate_amortization_schedule($1)',
[loanId]);

    await pgClient.query('COMMIT');

    // Update MongoDB with Audit Log
    await db.collection('credit_profiles').updateOne(
      { _id: new ObjectId(mongoProfileId) },
      {
        $set: {
          "underwriting_status": "APPROVED",
          "approved_loan_id": loanId,
          "updated_at": new Date()
        }
      }
    );

    return NextResponse.json({ success: true, message: 'Loan approved and
amortization schedule initialized.' });

  } catch (error: any) {
    await pgClient.query('ROLLBACK');
    return NextResponse.json({ error: error.message }, { status: 500 });
  } finally {
    pgClient.release();
  }
}
```

Role-Based Access Control & Dashboard UI Portals

The application enforces access controls across three user roles:

- **Customer Banking Portal (`/dashboard/customer`)**: Provides self-service banking operations for account holders. The UI uses client-side state management (SWR/React Query) for balance updates and transaction history polling. Core features include checking/savings balance cards, intra-bank funds transfer forms, loan application forms, and printable repayment schedules.
- **Loan Underwriter Review Board (`/dashboard/underwriter`)**: Designed for credit officers evaluating loan applications. Built using Server-Side Rendering (SSR) for data loading, combined with local Zustand state management for application evaluation. Key components include financial statement inspection views, interactive DSCR/OCF chart visualizers, credit risk score breakdowns, and approval/rejection decision controls.
- **Admin Risk Analytics Dashboard (`/dashboard/admin`)**: Serves executive leaders and risk admins managing system-wide exposure. The dashboard uses WebSockets to stream transaction volumes, non-performing loan (NPL) ratios, and aggregate credit exposure. Features include executive summary scorecards, account status controls (e.g., freezing accounts), and searchable audit logs generated by `account_audit_log`.

Alternative Web Framework Options

While Next.js offers a unified full-stack model, enterprise banking applications may also leverage Python-based frameworks like Django or Flask for specialized backend architectures.

Architectural Feature	Next.js (Node.js React Engine)	Django (Python Enterprise Monolith)	Flask (Python Microservice Framework)
System Architecture	Full-Stack Unified (SSG/SSR + Node API Routes).	Monolithic Backend Framework (Model-View-Template).	Lightweight Micro-framework.
Relational ORM Integration	Prisma, Drizzle, or native `pg` connection pools.	Native Django ORM with integrated schema migrations.	SQLAlchemy ORM with Alembic migration engine.
NoSQL / Mongo Driver	Native Node.js Driver (`mongodb`) or Mongoose.	PyMongo or MongoEngine document mappers.	PyMongo or Flask-PyMongo extensions.
Data Science Ecosystem	Moderate (Requires IPC/microservice bindings to Python risk engines).	Native access to `pandas`, `numpy`, `scipy`, `scikit-learn` libraries.	Native access to `pandas`, `numpy`, `scipy`, `scikit-learn` libraries.
Concurrency Infrastructure	Asynchronous single-threaded event loop.	Asynchronous Views (ASGI) alongside traditional WSGI.	WSGI / ASGI with Gevent or Quart async extensions.

Architectural Feature	Next.js (Node.js React Engine)	Django (Python Enterprise Monolith)	Flask (Python Microservice Framework)
Built-in Security Controls	CSRF protection, React XSS prevention, Middleware Auth.	Built-in ORM SQLi prevention, CSRF, XSS, Clickjacking protection.	Requires manual extension integration (Flask-SeaSurf, WTForms).

Framework Integration Scenarios

- **Django Paradigm**: Django is well-suited for traditional core banking applications that benefit from a single monolithic repository. Its built-in ORM handles complex relational schemas, while its native administration panel speeds up internal back-office tooling. Python's native data science ecosystem allows Django backends to process complex risk models directly within background task queues (e.g., Celery).
- **Flask Paradigm**: Flask is optimal for microservice-oriented architectures. In a hybrid deployment, a Flask service can operate as an isolated risk engine, executing heavy MongoDB aggregation pipelines, calculating DSCR metrics, and serving risk scores via REST APIs to a Next.js user interface.

6. Two-Member Team Work Breakdown Structure (WBS)

To ensure balanced contributions across database engineering and full-stack development, responsibilities are split into two distinct engineering workstreams.

Member 1: Database Architect & Core Banking Engineer

- **Primary Focus**: PostgreSQL Relational Engineering, Transaction Isolation, Core Ledger Engine, Transactional API Layer.

Development Phase	Engineering Deliverables
Phase 1: Relational Architecture	Design normalized 3NF ER database models for PostgreSQL. Write DDL scripts defining tables, foreign key policies, ENUM types, and CHECK constraints. Implement B-Tree and composite indexes optimized for operational throughput.
Phase 2: Database Business Logic	Implement the PL/pgSQL function `fn_generate_amortization_schedule` for compound payment generation. Write database triggers for automated auditing (`trg_account_balance_audit`) and minimum balance enforcement (`trg_validate_account_minimum_balance`).
Phase 3: Concurrency & Isolation	Write the PL/pgSQL procedure `sp_transfer_funds` featuring row-level locking (`FOR UPDATE`) to prevent deadlocks and race conditions. Construct analytical views (`vw_executive_portfolio_summary` , `vw_officer_loan_performance`).

Development Phase	Engineering Deliverables
Phase 4: Core APIs & Verification	Build transactional Next.js API route handlers (`/api/accounts` , `/api/transfers`) using `pg` connection pooling. Perform concurrency stress testing to verify deadlock prevention and rollback mechanics.

Member 2: Risk Systems & Full-Stack Application Engineer

- **Primary Focus**: MongoDB Schema Architecture, Dynamic Aggregations, Next.js UI/UX Dashboards, Role-Based Access Control.

Development Phase	Engineering Deliverables
Phase 1: NoSQL Document Store	Design polymorphic BSON schemas (`credit_profiles`) supporting Retail, SME, and Corporate borrower models. Implement JSON Schema validation rules and setup collection indexing.
Phase 2: Risk Aggregations	Write multi-stage MongoDB Aggregation Pipelines to compute real-time DSCR ratios, TTM Operating Cash Flow margins, and weighted credit risk scores from nested arrays.
Phase 3: Application UI & RBAC	Build responsive web interfaces for Customer Banking, Underwriter Review, and Admin Analytics portals using Next.js and Tailwind CSS. Implement RBAC session middleware enforcing route permissions across user roles.
Phase 4: System Orchestration	Integrate the Next.js frontend with MongoDB analytics via a singleton `MongoClient` connection. Build cross-database workflows connecting MongoDB underwriting outputs to PostgreSQL loan generation routines.

7. System Verification & Operational Acceptance

Relational Transaction Test Script

The following transaction test verifies isolation guarantees and rollback handling during concurrent debit operations.

SQL

```
-- Transaction Concurrency & Rollback Test
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Simulate account transfer
CALL sp_transfer_funds(
```

```
    'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11'::UUID,
    'b1ffcd00-0d1c-5ff9-cc7e-7cc0ce491b22'::UUID,
    500.00,
    'Concurrency Validation Test',
    NULL
);

-- Verify that resulting balance adheres to minimum balance thresholds
-- If threshold is breached, trigger 'trg_validate_account_minimum_balance'
raises exception 23514 and aborts transaction.

COMMIT;
```

MongoDB Aggregation Test Verification

The command below validates pipeline execution against the `credit_profiles` collection to ensure accurate real-time metric calculation.

JavaScript

```
// Verification query for target credit profile evaluation
db.credit_profiles.find(
  { "borrower_id": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11" },
  { "legal_name": 1, "financial_statements": 1, "credit_bureau_payload": 1 }
).pretty();
```

This verification plan confirms that transactional operations maintain strict consistency within PostgreSQL, while MongoDB processes unstructured financial profiles to deliver real-time risk scores.